

* Interrupts :-

→ A single processor can serve many peripheral devices. There are two ways to do that: 1) Interrupt method 2) polling method.

→ Interrupt method:

- whenever a device needs service, the device notifies the CPU by sending an Interrupt signal.
- upon receiving an Interrupt signal, the CPU stops its job doing and serves the device.
- This technique facilitates the effective utilisation of CPU timings.

→ polling:

- The processor continuously monitors the status of a given device.
- When the status condition is met, it performs the service.
- After that, it moves on to monitor the next device until each one is serviced.
- This method wastes most of the CPU's time by polling devices when they do not need service.

Note:

Polling

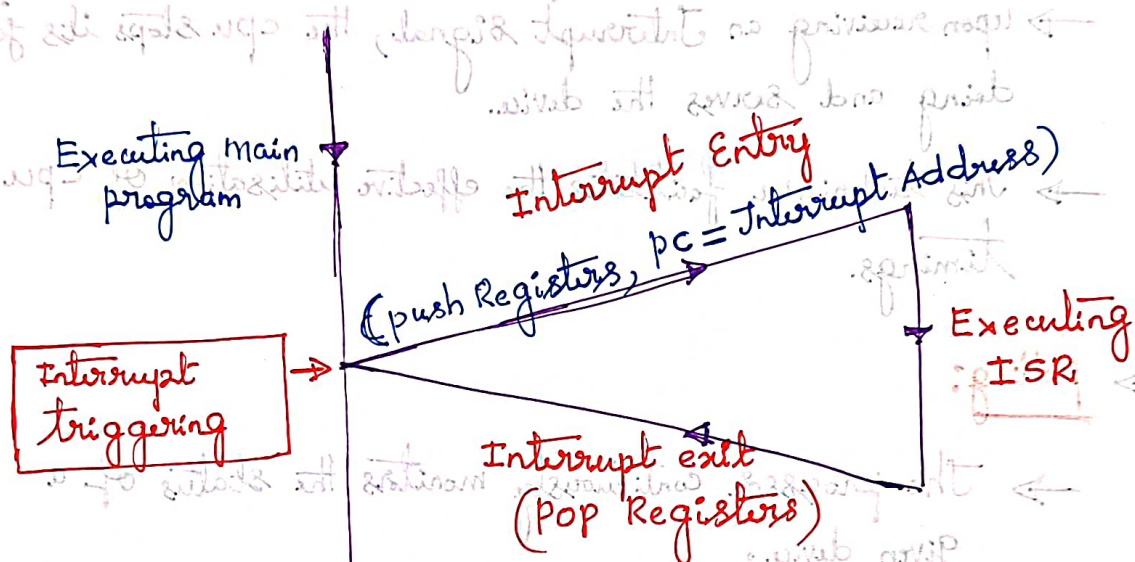
- Generated by processor, ask device if it needs attention/service
- Happens regularly
- It's not an efficient use of the CPU time.

Interrupts

- Generated by peripheral device
- It can happen at any time.
- It's an efficient use of the CPU time.

* Interrupt Service Routine (ISR) :-

- For every Interrupt, there is a program associated with it.
- When an Interrupt occurs this program is executed to perform some service for the Interrupt. This program is commonly referred to as an Interrupt service routine (ISR) or an Interrupt handler.



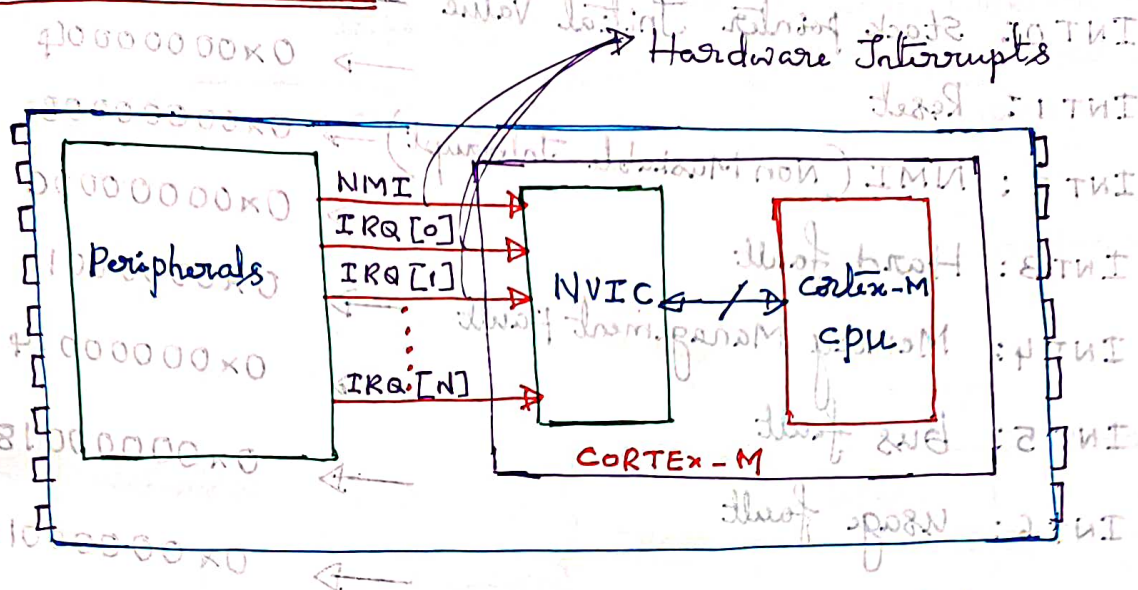
Executing main program

Note

Interrupt

Handler

* NVIC In cortex-M



- As shown in the above figure, there are some pins which are associated with Hardware Interrupts.
- Hardware Interrupts are input signals to the processor.
- When the input pins are triggered, the processor pushes the content of program counter (PC) register value into stack and loads the PC register with the address of the Interrupt Service Routine. And, ISR get executed.

* Interrupt Vector Table:

- Each Interrupt is associated with program called ISR.
- Interrupt vector table is a look-up table which holds the addresses of ISRs (Interrupt Service Routines).
- In the ARM cortex-M4, the lowest 1024 bytes ($256 \times 4 = 1024$) of memory space are set aside for the Interrupt Vector table and this space is not used for any other functions.

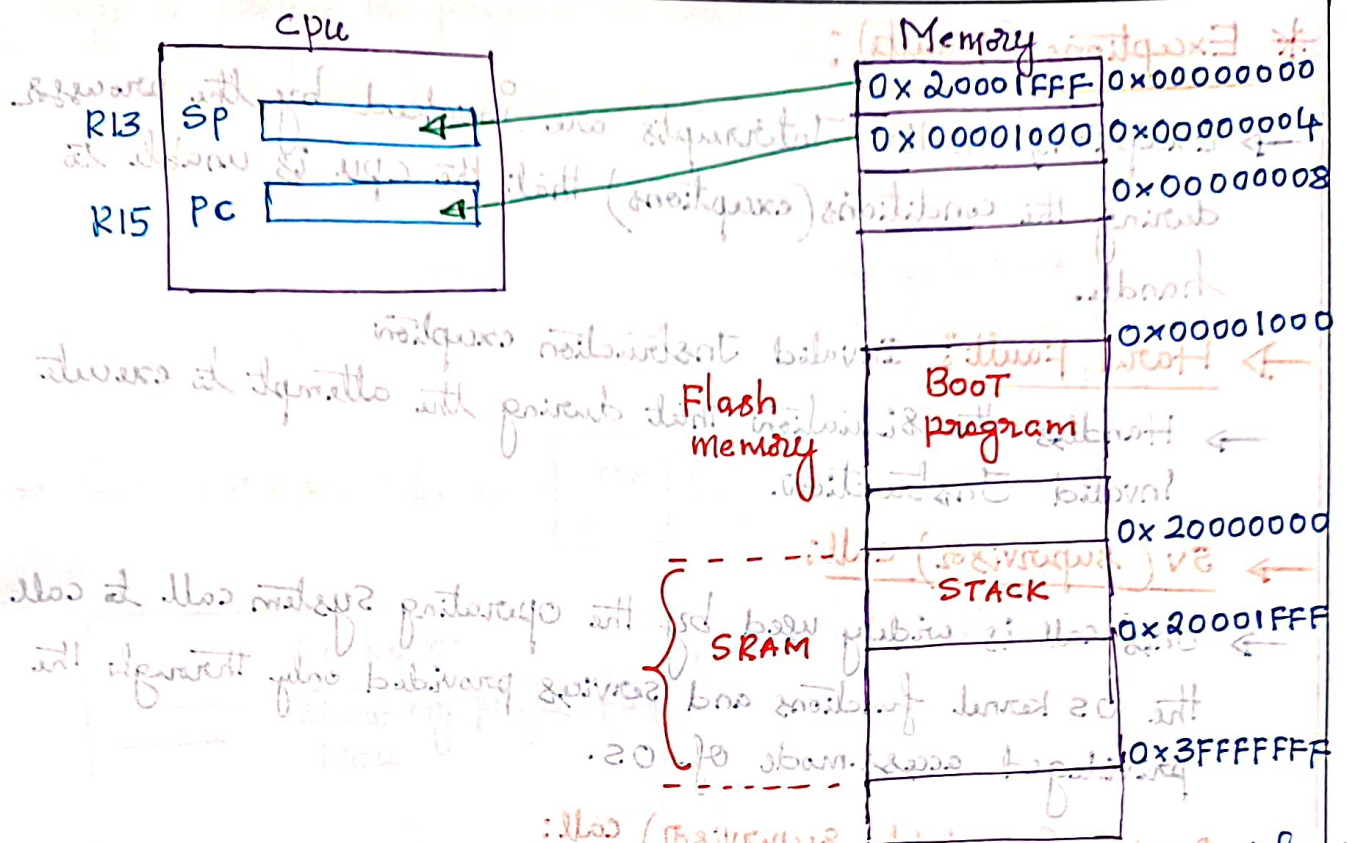
NOTE: The Interrupt Vector Table is located at the lowest 1024 bytes of memory space.

ARM Cortex-M4 Interrupt Vector Table

* List of Interrupts and their designated functions:

INT0: Stack pointer Initial value	→ 0x00000000
INT1: Reset	→ 0x00000004
INT2: NMI (Non Maskable Interrupt)	→ 0x00000008
INT3: Hard fault	→ 0x0000000C
INT4: Memory Management fault	→ 0x00000010
INT5: Bus fault	→ 0x00000014
INT6: Usage fault	→ 0x00000018
INT7: Reserved	→ 0x0000001C
INT8: Reserved	→ 0x00000020
INT9: Reserved	→ 0x00000024
INT10: Reserved	→ 0x00000028
INT11: SV call (SVC: Supervisor call)	→ 0x0000002C
INT12: Reserved	→ 0x00000030
INT13: Reserved	→ 0x00000034
INT14: PendSV (pendable Supervisor call)	→ 0x00000038
INT15: SysTick	→ 0x0000003C
INT16: IRQ0 for peripherals	→ 0x00000040
INT17: IRQ1 for peripherals	→ 0x00000044
⋮	
INT255: IRQ239 for peripherals	→ 0x000003FC

NOTE: The Interrupts INT0 to INT15 are used for internal operations.



→ At the reset mode, the CPU goes to known state and the predefined values are loaded to the registers.

* Non-Maskable Interrupt:

→ **Hardware Interrupts:** IRQs (Interrupt request) and NMI (Non-maskable Interrupt) are hardware Interrupts.

→ **IRQ:** is an Input signal to CPU which can be masked (ignored) and unmasked through the use of software.

→ **NMI:** is an Input signal to CPU which cannot be masked by software. Hence it is called as "Non-maskable Interrupt".

→ In ARM cortex-M NVIC, INT-2 has assigned to NMI. When the NMI pin is activated, the CPU control will jump to location 0x00000008 to get the address of the Interrupt service routine (ISR) associated with NMI.

→ Memory locations 0x00000008 to 0x0000000B contains the 4 bytes of address associated with the ISR belonging to NMI.

* Exceptions (Faults):

→ Exceptions/faults Interrupts are invoked by the processor during the conditions (exceptions) that the CPU is unable to handle.

→ Hard Fault: Invalid Instruction exception
→ Handles the situation that during the attempt to execute Invalid Instructions.

→ SV (supervisor) call:

→ This call is widely used by the operating system call to call the OS kernel functions and services provided only through the privileged access mode of OS.

→ Pend SV (pendable supervisor) call:

→ This call used by OS at the time of handling priority Interrupts

→ SysTick: This call is being used in the environment of multitasking OS.

→ IRQ peripheral Interrupts: Interrupts used for peripherals.

* NOTE:

→ Many Interrupts are asynchronous. Interrupt may occur at any time during the program execution.

* Context switching / context saving:

→ The saving of processor contents before switching to Interrupt handler/ISR is called as context switching / context saving.

→ The ARM cortex-M automatically saves the contents of registers (CPSR, PC, LR, R12, R3, ..., R0) on to the stack when an interrupt is acknowledged.

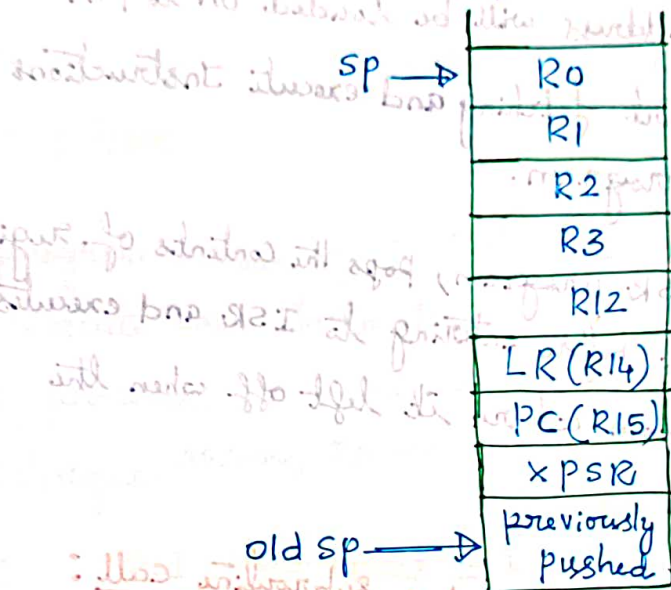
* Processing any Interrupts in Arm cortex-M:

→ Steps to be followed

→ Stack frame upon Interrupt:

NOTE:

xPSR → program status
Register
A: Application
E: Execution
I: Interrupt



Decreasing memory
address

STACK

1. The current program status register (CPSR) is pushed onto the stack and SP is decremented by 4 since xPSR is a 4-byte Register.
2. The current PC (R15) is pushed on to the stack and SP is decremented by 4.
3. The current LR (R14) is pushed onto the stack and SP is decremented by 4.
4. The current R12 is pushed onto stack and SP is decremented by 4.
5. The current R3 is pushed onto stack and SP is decremented by 4.
6. The current R2 is pushed onto stack and SP is decremented by 4.
7. The current R1 is pushed onto stack and SP is decremented by 4.
8. The current R0 is pushed onto stack and SP is decremented by 4.

9. The CPU goes into the Interrupt handler mode.
10. The INT number (type) is multiplied by 4 to get the offset of the location within the Vector table to fetch the address of the ISR. And, the ISR address will be loaded on to PC.
11. The processor will start fetching and execute Instructions belongs to ISR program.
12. After execution of ISR program, pops the contents of registers which saved in stack before entering to ISR and executes the main program from where it left off when the interrupt occurred.

* Difference between Interrupt and a Subroutine call :

→ Types of Branch Instructions:

- B : Branch
- BL : Branch with link : This instruction is used to call subroutine.
- BX : Branch exchange
- BLX : Branch exchange with link

* Interrupt

- Before entering to ISR, saves the address of the next Instruction on to the stack
- Goes to a fixed memory location in the Interrupt Vector table to get the address of the ISR
- INT belonging to externally activated hardware interrupts can be masked.
- Interrupt puts the CPU in the Handler mode

* Subroutine/procedure

- Before entering to subroutine, saves the address of the next Instruction on to Link Register (LR; R14)
- BL Instruction can take an argument and jump to any location within the 4GB address range.
- BL instruction cannot be masked (disabled).
- BL Instruction does not change the CPU execution mode.

* Interrupt priority:

→ In the ARM cortex-M, the Reset, NMI and hard fault exceptions have fixed priority levels.

→ RESET
NMI
HARD FAULT

↑ Highest priority
↓ Lowest priority

} If all three of them are activated at the same time, the Reset Interrupt will be executed first.

→ NMI
IRQ

↑ Highest priority
↓ Lowest priority

} If both NMI and IRQ are activated at the same time, NMI will be executed first.

→ The remaining exceptions and IRQs have lower priority and are configurable. Means that priority levels can be configured by the programmer.

NOTE : When a low priority interrupt is being executed, a higher priority interrupt can preempt it.

* ARM cortex-M processor modes:

→ The ARM cortex-M can run in one of the two modes at any point of time. They are

1) Thread (Application) mode:

2) Handler (exception) mode:

→ Thread (application) mode : Its a normal mode, when the processor is powered on, it automatically goes to the Thread mode.

→ Handler (exception) mode : processor executes ISR in this mode. upon returning from ISR, the CPU automatically changes from Handler mode back to Thread mode.

→ out of all interrupts, only the RESET forces the CPU into Thread mode and the remaining interrupts are executed in Handler mode.

* There are two stacks in ARM cortex:

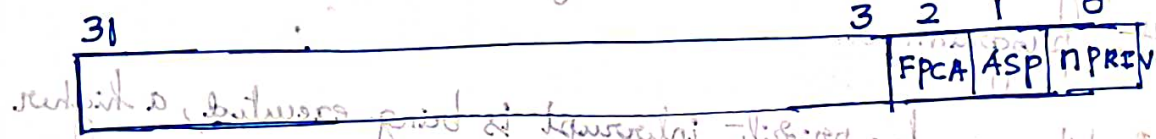
→ The ARM cortex-M processor has two stack pointer registers

1) PSP (processor stack pointer)

2) MSP (Main stack pointer)

* Control Register:

The Control Register (CR) is a 32-bit register that contains control bits for the processor. It is used to configure the processor's behavior during execution.



BIT-0: PRIV → Defines the privilege level

0: privileged

1: unprivileged

ASP: Active stack pointer → Defines the currently active stack pointer

0: MSP is the current stack pointer

1: PSP is the current stack pointer

FPCA: Floating point context Active

0: No floating point context Active

1: Floating point context Active.

* programming PWM in STM ARM *

* PWM with Timer *

→ We have to ^{enable} ~~set~~ the one of ^{the} Timers to set the PWM generation

→ Example: TIM1

clock source: Internal clock

channel1: PWM Generation CH1

channel2: Disable

channel3: Disable

channel4: Disable

→ Enable pin PE9 as PWM output [CH1]

→ Go to configuration window to set the PWM frequency and duty cycle.

→ click Timer-1 (TIM1)

Counter settings:

prescaler (psc - 16 bits value): 16 [16 MHz]

counter period (Auto Reload) : 100 → frequency value

PWM Generation channel 1

pulse (16 bits value) : 50 [Duty cycle setting 50%]



NOTE: The PWM signal has two key parameters: Frequency & duty cycle.

The PWM signal is used to control the speed of motor. (DC Motor)

```
#include "main.h"
```

```
/* Private Variable */
```

```
TIM_HandleTypeDef htim1;
```

```
int main(void)
{
```

```
/* Initialize all configured peripherals */
```

```
Mx_GPIO_Init();
```

```
Mx_TIM_Init();
```

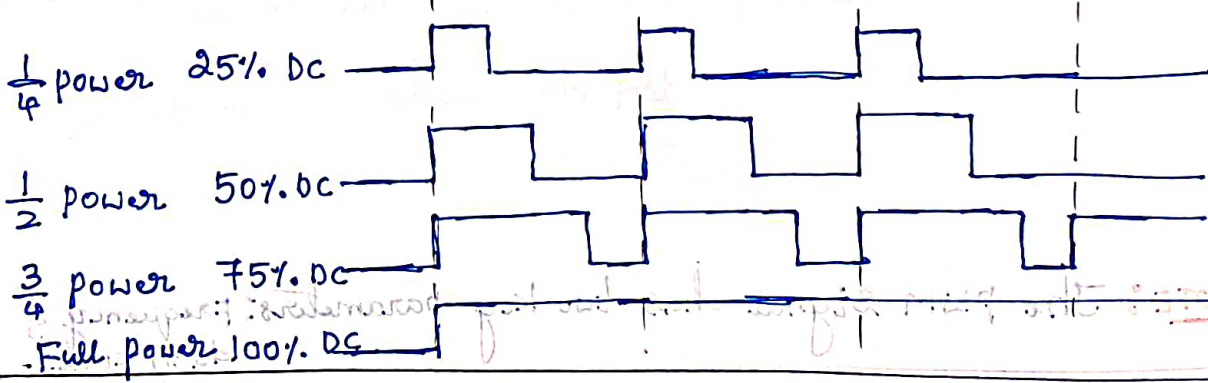
```
/* USER CODE BEGIN */
```

```
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
```

```
while(1)
{
```

```
    // Your code here
}
```

```
/* pulse width Modulation comparison
```



The PWM signal is used to control the speed of motor (DC motor).